Hafta 7 — Derinlemesine

> ## ⚠️ BU DOSYA SINAV KAPSAMI DIŞINDADIR
>
> Buradaki hiçbir konudan sınavda soru sorulmaz. Bu dosya, *"peki bu nasıl çalışıyor?"* diye soran öğrenci içindir. Sınav kapsamı `H07-ders-notu.md` dosyasıdır.
>
> Dönem projesinde işinize yarayacak birkaç şey de buradadır (bölüm 1, bölüm 2, bölüm 5).
>
> "Ölçüldü" yazan satırlar `Hafta-07/Bitmis` projesinin bir **kopyasında** denendi; projenin kendisi değiştirilmedi. Ölçülmeyen bölümler bunu açıkça söyler.

---

1. Cookie'yi İmzalamak: Data Protection

Ders notunun Klasik Tuzak'ında rolü session'a taşıdık. Bazen değeri yine de cookie'de tutmak istersiniz — örneğin sunucuda hiçbir şey saklamadan. O zaman değer **imzalanır**: kullanıcı onu görebilir ama değiştirirse sunucu anlar.

ASP.NET Core'un kendi session ve anti-forgery cookie'leri bunu zaten yapar; kullandığı araç **Data Protection**'dır ve sizin kodunuza da açıktır:

```
using Microsoft.AspNetCore.DataProtection;

public class DurumController : Controller
{
    private readonly IDataProtector _koruyucu;

    public DurumController(IDataProtectionProvider saglayici) =>
        _koruyucu = saglayici.CreateProtector("KampusKitap.Rol");   // amaç adı

    public IActionResult Giris()
    {
        Response.Cookies.Append("RolImzali", _koruyucu.Protect("ogrenci"));
        return RedirectToAction(nameof(Panel));
    }

    public IActionResult Panel()
    {
        var rol = "misafir";
        try { rol = _koruyucu.Unprotect(Request.Cookies["RolImzali"] ?? ""); }
        catch (System.Security.Cryptography.CryptographicException) { /* değiştirilmiş */ }
        return View(model: rol);
    }
}
```

Ölçülen:

Cookie'deki değer	`Unprotect` sonucu
`Protect("ogrenci")` çıktısı (134 karakter, `CfDJ8A4-mf96z…`)	`ogrenci`
Elle `yonetici` yazıldı	`CryptographicException` : *The provided payload cannot be decrypted because it was not protected with this protection provider.*
Ortadaki tek bir karakter değiştirildi	`CryptographicException` : *The payload was invalid.*

`Protect` hem **şifreler** (kullanıcı `ogrenci` sözcüğünü göremez) hem **imzalar** (değiştirirse çözülemez). `CreateProtector`'a verilen amaç adı farklı yerlerin birbirinin değerini çözmesini engeller.

> ⚠️ Ölçüm notu: imzalı değerin **son** karakteri değiştirildiğinde değer yine çözüldü. Bu bir açık değildir: base64'ün son karakteri bazen anlamsız dolgu bitleri taşır ve değiştirmek çözülen baytları değiştirmez. Ortadaki bir karakter imzayı bozar.

İmzalamak değerin **değiştirilmediğini** garanti eder, **güncel** olduğunu garanti etmez: kullanıcı eski ama geçerli bir imzalı cookie'yi saklayıp geri koyabilir. Rolü kaldırılan bir kullanıcının eski cookie'si çalışmaya devam eder. Session'da bu sorun yoktur: sunucu kaydı siler. Hafta 11'in kimlik cookie'si bu yüzden bir süreyle ve gerektiğinde sunucu tarafı denetimle kullanılır.

---

2. `TempData`'yı Session'da Tutmak

Hafta 4'ten beri `TempData` bir cookie ile taşınıyor (`.AspNetCore.Mvc.CookieTempDataProvider`). Session kurulu bir projede onu sunucuya taşımak tek satırdır:

```
builder.Services.AddControllersWithViews()
    .AddSessionStateTempDataProvider();
// AddSession ve UseSession da gerekir (ders notu bölüm 3.2–3.3)
```

Ölçülen: aynı `TempData["Not"] = "merhaba"; return RedirectToAction(...)` için

Sağlayıcı	Yönlendirme response'undaki `Set-Cookie`	Okunan
Varsayılan (cookie)	`.AspNetCore.Mvc.CookieTempDataProvider=CfDJ8A4-m…`	`merhaba`
Session	`.AspNetCore.Session=CfDJ8A4%2Bmf96z…`	`merhaba`

Ne zaman tercih edilir? `TempData`'ya konan veri büyükse (cookie ~4 KB'ta durur) ya da kullanıcının görmemesi gerekiyorsa. Bedeli: session'ın bütün sınırları (bellek, tek sunucu, yeniden başlatma) artık `TempData` için de geçerlidir.

---

3. Dağıtık Session: Redis ve SQL Server

Bellek içi session tek sunucuda çalışır (ders notu bölüm 3.7). Uygulama iki sunucuya dağıtılırsa, bir request'i karşılayan sunucunun belleğindeki session ötekinde yoktur. Çözüm, session verisini iki sunucunun da eriştiği bir yere koymaktır.

ASP.NET Core'da session, `IDistributedCache` arayüzü üzerinden çalışır. Bu yüzden değişen tek şey o arayüzü kaydeden satırdır; `AddSession`, `UseSession` ve `NesneYaz` / `NesneOku` olduğu gibi kalır:

```
// Redis — paket: Microsoft.Extensions.Caching.StackExchangeRedis
builder.Services.AddStackExchangeRedisCache(o =>
{
    o.Configuration = builder.Configuration.GetConnectionString("Redis");
    o.InstanceName = "KampusKitap:";
});

// SQL Server — paket: Microsoft.Extensions.Caching.SqlServer
builder.Services.AddDistributedSqlServerCache(o =>
{
    o.ConnectionString = builder.Configuration.GetConnectionString("Varsayilan");
    o.SchemaName = "dbo";
    o.TableName = "SessionVerisi";
});
```

Bu satırlardan biri `AddDistributedMemoryCache`'in yerine geçer. Ders notunda "bu satırı yine de yazıyoruz" dememizin sebebi buydu.

> Bu bölüm ölçülmedi; kod Microsoft Learn'deki *Distributed caching in ASP.NET Core* sayfasından uyarlandı. İki sunuculu bir dağıtım Hafta 14'ün konusu olabilir.

Bir ayrıntı daha: session cookie'si **Data Protection** ile şifrelenir (bölüm 1). İki sunucu, birinin yazdığı cookie'yi ötekinin çözebilmesi için **aynı anahtarları** paylaşmalıdır (`AddDataProtection().PersistKeysTo…`). Paylaşmazsa her sunucu ötekinin cookie'sini tanımaz ve kullanıcının session'ı her request'te sıfırlanır.

---

4. Session'ın İçi

Microsoft Learn'deki *Session and state management* sayfasının üç ayrıntısı (ölçülmedi):

- **Kilitlenmez.** Aynı kullanıcının iki request'i aynı anda session'ı değiştirirse **son yazan kazanır**. İki sekmeden aynı anda sepete ekleyen kullanıcının bir eklemesi kaybolabilir. Kritik veri (sipariş, ödeme) bu yüzden session'da değil veritabanında tutulur.
- **Yükleme.** Session verisi ilk erişimde, request'i bekleterek yüklenir. Dağıtık önbellekte bu bir ağ çağrısıdır. Beklememek için action'ın başında `await HttpContext.Session.LoadAsync();` çağrılabilir.
- **Yazma.** Değişiklikler request bitince **bir kez** önbelleğe yazılır. Ders notunda ölçtüğümüz "session cookie'si ilk yazmada gelir" davranışı da buradan gelir: hiçbir şey yazılmadıysa ne kayıt açılır ne cookie gönderilir.

---

5. `SameSite`'ı İki Siteyle Ölçmek

Ders notunda `SameSite`'ın başka siteden başlayan request'leri sınırladığını söyledik. Bunu ölçmek için iki **site** gerekir: `localhost` ile `127.0.0.1` tarayıcı için iki ayrı sitedir, aynı uygulamaya gitseler bile.

Deney: `localhost`'ta dört cookie yazıldı — `ss-lax` (`Lax`), `ss-strict` (`Strict`), `ss-none` (`None` + `Secure`), `ss-yok` (`SameSite` yazılmadı). Sonra `127.0.0.1`'deki bir sayfadan `localhost`'a bir bağlantı tıklandı ve bir form gönderildi. Sunucuya gelenler (ölçüldü, Chrome):

Request	`ss-lax`	`ss-strict`	`ss-none`	`ss-yok`
Aynı siteden GET	✓	✓	✓	✓
Başka siteden **bağlantı** (üst düzey GET)	✓	—	✓	✓
Başka siteden **form POST** — cookie 2 dakikadan yeni	—	—	✓	✓
Başka siteden **form POST** — cookie 2 dakikadan eski	—	—	✓	—

Üç sonuç:

1. `Lax` **POST'u keser.** CSRF'in klasik biçimi (başka sitedeki görünmez form) `Lax` cookie'yi taşıyamaz. ASP.NET Core session cookie'sini bu yüzden `samesite=lax` yazar.
2. `Strict` **bağlantıyı bile keser.** Başka bir siteden gelen kullanıcı ilk sayfada "giriş yapmamış" görünür. Bankacılık gibi uygulamalar için uygundur; bir alışveriş sitesinde kullanıcıyı şaşırtır.
3. `SameSite` **yazılmayan cookie ilk iki dakika POST'ta gider.** Chrome böyle bir cookie'ye `Lax` gibi davranır, ama yeni yazılmış cookie'ler için iki dakikalık bir istisna tanır (*Lax-allowing-unsafe*). Sonuç: `SameSite`'ı yazmamak, tam olarak `Lax` yazmak değildir. Önemli cookie'lerde **açıkça** yazın — `SonBakilan`'da yaptığımız gibi.

> `SameSite` tarayıcının uyguladığı bir kuraldır; eski ya da farklı tarayıcılar farklı davranabilir. Bu yüzden CSRF'e karşı asıl savunma Hafta 5'teki anti-forgery token'dır (`H05-detay.md` bölüm 4).

---

6. Cookie Önekleri: `__Host-` ve `__Secure-`

Cookie adının başına konan iki önek, tarayıcıya ek kurallar dayatır (MDN; ölçülmedi):

Önek	Tarayıcı ancak şu durumda kabul eder
`__Secure-`	Cookie `Secure` ile ve HTTPS üzerinden yazılmışsa
`__Host-`	`Secure` + HTTPS + `Path=/` + `Domain` **yok** (yalnızca bu tam ana makine)

`__Host-` öneki, bir alt alan adının (ör. `kotu.kampus.edu.tr`) ana alan adı için cookie yazmasını engeller. ASP.NET Core'da yalnızca adı değiştirmek yeter:

```
builder.Services.AddSession(o =>
{
    o.Cookie.Name = "__Host-KampusKitap.Session";
    o.Cookie.SecurePolicy = CookieSecurePolicy.Always;
});
```

---

7. `localStorage` ve `sessionStorage`

Tarayıcıda state'in iki yeri daha vardır ve ikisi de **JavaScript'e** aittir (MDN; ölçülmedi):

	Cookie	**`localStorage`**	**`sessionStorage`**
Sunucuya kendiliğinden gider mi?	**Evet**, her request'te	Hayır	Hayır
Kim okur?	Sunucu; `HttpOnly` değilse JavaScript	Yalnızca JavaScript	Yalnızca JavaScript
Ne kadar yaşar?	`Expires`'a göre	Silinene kadar	Sekme kapanana kadar
Boyut	~4 KB	~5 MB	~5 MB

İki sonuç:

- `HttpOnly`**'nin karşılığı yoktur.** Sayfaya sızan bir betik `localStorage`'daki her şeyi okur. Session anahtarı ya da kimlik bilgisi bu yüzden `localStorage`'a konmaz.
- **Kullanıcının elindedir.** Ders notunun Klasik Tuzak'ı aynen geçerlidir: "yönetici mi?" bilgisini `localStorage`'da tutmak, cookie'de tutmakla aynı hatadır.

Tema tercihi gibi yalnızca tarayıcının ilgilendiği veriler için uygundur: sunucunun bilmesi gerekmiyorsa her request'e eklenmesine de gerek yoktur.

---

8. Daha İleri Gitmek İsteyenlere

Konu	**Nereden başlanır**
Cookie onayı (KVKK/GDPR)	`CookiePolicyOptions`, `UseCookiePolicy`, `CookieOptions.IsEssential`
Cookie ile kimlik doğrulama	**Hafta 11** — `AddAuthentication().AddCookie()`
Response caching ve cookie	`Cache-Control: private`; cookie içeren response'lar paylaşımlı önbelleğe girmemeli
Session fixation	Girişte session'ı yenilemek: OWASP Session Management Cheat Sheet
Çok sunuculu dağıtımda Data Protection anahtarları	`PersistKeysToFileSystem`, `PersistKeysToStackExchangeRedis`

Kaynaklar: Session and state management in ASP.NET Core · Distributed caching in ASP.NET Core · ASP.NET Core Data Protection overview · Work with SameSite cookies · MDN — Using HTTP cookies